

미니프로젝트 2차 조별 발표

AI 10반 26조



목차

Content 1

Summary

Content 2-1

프로젝트 진행 & 분업

Content 2-2

결과

Content 2-3

개선점

Content 3

부록

1.Summary

목표 AI 기반 강의 생성 및 자동 영상 제작 Agent 시스템

- 주요 기능**
- 프레젠테이션 자료 분석
 - 분석 기반 강의 스크립트&음성&영상 생성
 - gradio 웹 서비스 제공

- 특징**
- 강의 콘텐츠 자동 생성 (Text → Script → Voice → Video)
 - 강의 흐름을 이해하는 Ai Agent 구조 설계
 - 웹 기반 인터페이스로 사용자 접근성 확보

- 기대 효과**
- 사내 교육의 자동화 >
즉시성(빠른 반영), 효율성(인력 비용절감),
표준화(품질의 균일화)

Our team's goal

- 효율적인 협업 체계 구축을 통한 프로젝트 생산성 극대화

2-1. 프로젝트 진행 & 분업



Top-Down 방식 채택

[노드 함수 작성 규칙 설정]

노드 함수를 구현하는 과정을 단계별로 정의. 각 조원은 정의된 단계를 참고하여 노드 함수를 작성.

1. 스테이트 전체 구조 통일

2. 구체적인 입력(Input-State)과 출력(Output-State) 정확하게 설정

3. 내부 로직 구분 (llm.invoke 사용 유무)

4. 노드 함수 흐름 반영 (그래프상 해당 노드의 위치 확인)

[노드 함수 작성 흐름]

데이터 준비 구문 작성



메시지 구성



llm.invoke() 호출

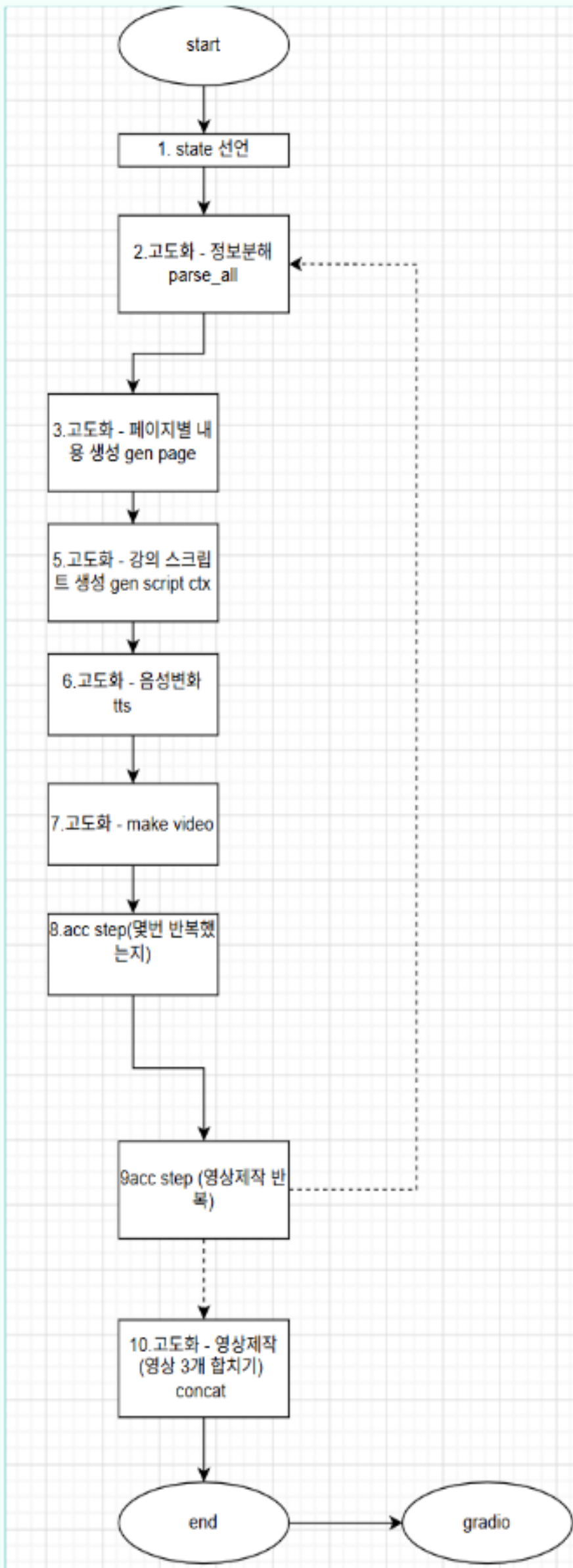


코드 결과로 State 업데이트

역할

작성 노드 함수

이승호	조장, 발표자 / State 관리자	영상제작
손가영	발표자, 검토담당자	강의 스크립트생성
김유찬	PPT 제작 / Graph 관리자	정보분해
이채현	PPT 제작	입력 프롬프트
윤명세	PPT 제작	페이지별 내용 생성
박승원	타임키퍼	음성변환
류연우	서기 / Graph 관리자	정보분해
박주영	검토담당자	반추



2-2. 결과

[완성된 동영상]

모델 성능 모니터링

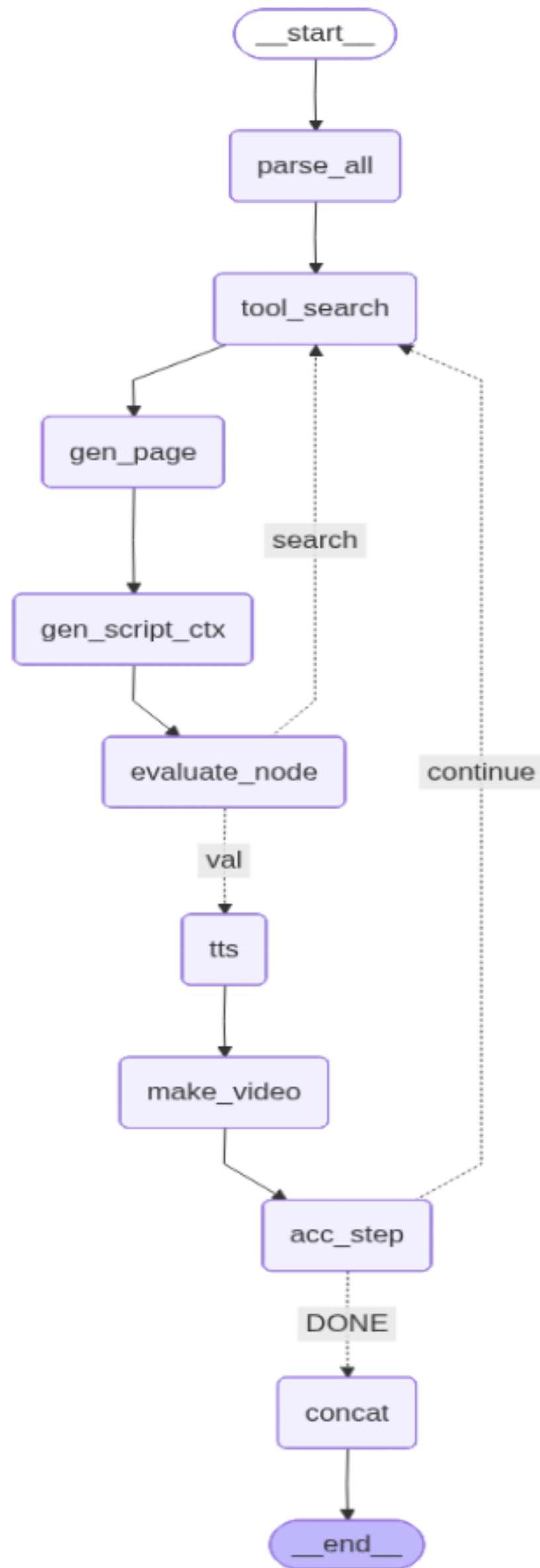
✓ 모델 모니터링

- 프로덕션 환경에서 ML 모델의 성능 및 동작을 지속적으로 추적, 분석, 평가하는 프로세스
- 필요성 : 모델은 배포 후에도 품질 저하, 데이터 문제, 사용 환경 변화 등의 위험에 노출됨
- 주요 리스크
 - 데이터/컨셉 드리프트
 - 데이터 품질 문제
 - 적대적 공격(예: prompt injection)
 - 연결된 앞 단계의 모델 오류 전파
- 모니터링 목표
 - 문제 조기 탐지
 - 근본 원인 분석
 - 모델 동작 이해 및 투명한 문서화



1

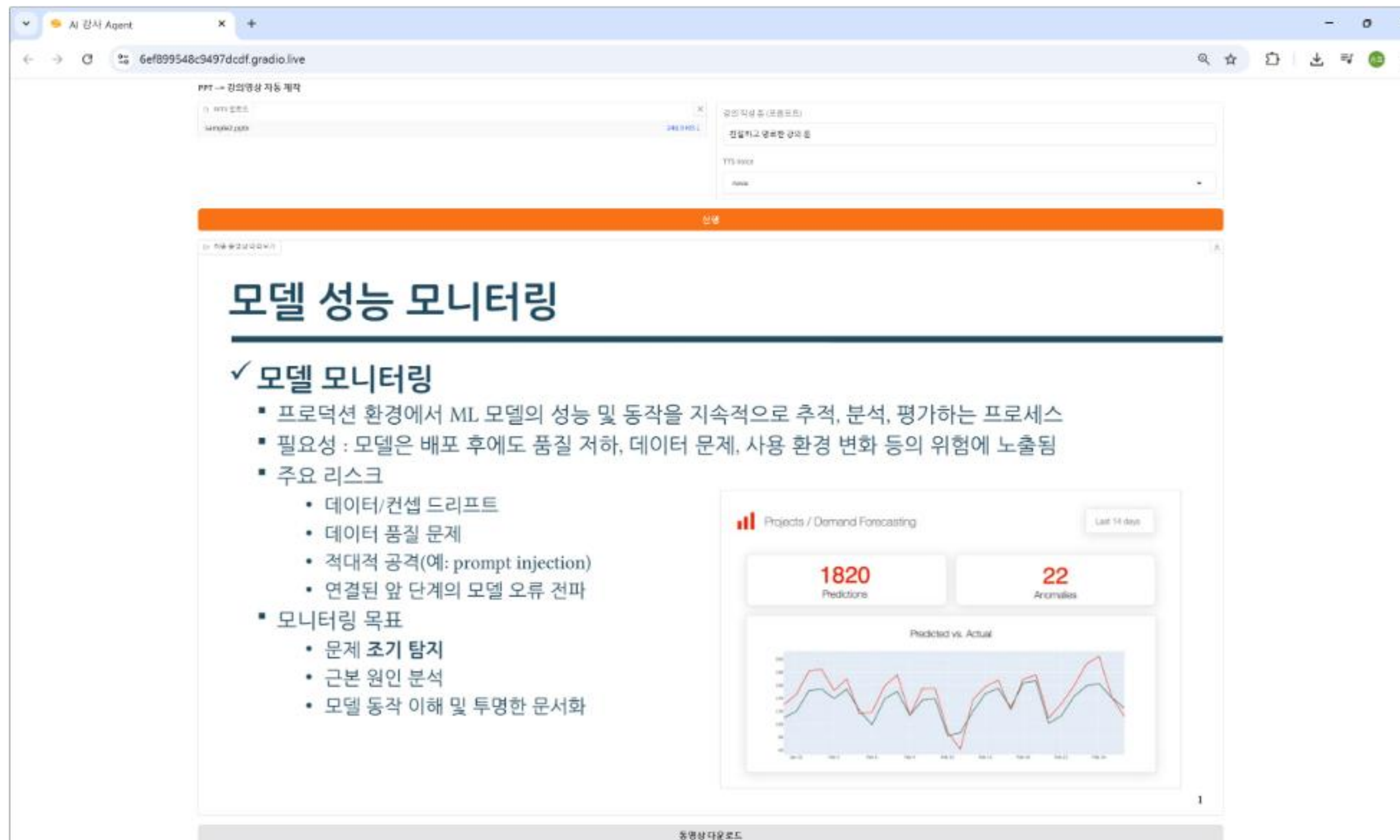
https://drive.google.com/file/d/1sDYP9dHI1nAaL89XhCNq_7-v_nBavM_/view?usp=sharing



2-2. 결과

추가 기능 구현 - 반추

■ gradio 웹 서비스



반추 기능

AI가 생성한 스크립트를 스스로 평가
잘못된 정보 혹은 부족한 설명을 채워 자동으로 성능 향상

```
from langchain_core.prompts import ChatPromptTemplate

def evaluate_node(state: State):
    print("--- 검증 중... ---")

    script = state["cur_script"]
    context = state["cur_page_content"]
    current_retry = state.get("retry_count", 0) # 현재 횟수 가져오기

    eval_prompt = ChatPromptTemplate.from_template("""
    당신은 품질 관리 전문가입니다. 아래 내용을 바탕으로 스크립트를 평가하세요.
    [검색된 정보]: {context}
    [작성된 스크립트]: {script}

    평가 기준:
    1. 스크립트가 검색된 정보를 충분히 반영하고 있는가?
    2. 정보가 부정확하거나 거짓(Hallucination)이 섞여있지 않은가?

    반드시 답변 시작에 'PASS' 혹은 'FAIL'을 적어주세요.
    """)

    chain = eval_prompt | llm
    response = chain.invoke({"context": context, "script": script})
    result = response.content.upper()

    if "PASS" in result:
        print("검증 통과!")
        return {
            "is_valid": True,
            # 성공했으므로 재시도 횟수를 올리지 않음
            "retry_count": current_retry
        }
    else:
        print(f"검증 실패: {result[:100]}...")
        # 실패해서 다시 돌아갈 것이므로 카운트 +1
        return {
            "is_valid": False,
            "retry_count": current_retry + 1
        }
```

2-3. 개선점

1. 정확도

슬라이드 내용을 기반으로 스크립트를 생성하지만, 일부 페이지에서는 문맥을 잘못 이해하거나 부정확한 설명이 생성되는 경우가 있음.
→ 향후 전체 슬라이드 흐름을 반영한 **문맥 기반 생성**으로 개선 가능.

2. 사진 분석 기능 강화

이미지 위주의 슬라이드나 손글씨 사진이 포함된 경우, 내용을 정확히 인식하지 못하는 한계가 있음.
→ **OCR 및 Vision 모델**을 추가하여 이미지 이해 성능 향상 가능.

3. 자막 기능 추가

생성된 강의 영상에 자막 기능이 없어 가독성과 접근성이 다소 부족함.
→ **음성 기반 자동 자막 생성 기능** 추가 시 사용자 편의성 향상 기대.

3. 부록 - 전체 코드

[State 선언]

```

1 class State(TypedDict, total=False):
2     # 입력, 설정
3     pptx_path: str
4     work_dir: str
5     prompt: Dict[str, Any]
6
7     # 파싱 결과
8     slides: List[Dict[str, Any]]
9     n_slides: int
10    slide_index: int
11
12    # 생성, 미디어 누적 산출물
13    page_contents: List[str]
14    scripts: List[str]
15    audios: List[str]
16    videos: List[str]
17    final_video: str
18    retry_count: int
19    is_valid: bool
20
21    # 임시
22    cur_search_context: str
23    cur_page_content: str
24    cur_script: str
25    cur_audio: str
26    cur_video: str

```

[정보분해]

```

def node_parse_all(state: State) -> State:
    '파서 노드 내부에서 반복문으로 모든 슬라이드 텍스트/표/이미지 + 스냅샷 생성'
    pres = Presentation(state["pptx_path"])
    work_dir = state["work_dir"]
    media_dir = os.path.join(work_dir, "media")
    os.makedirs(media_dir, exist_ok=True)

    slides_out: List[Dict[str, Any]] = []

    for idx, slide in enumerate(pres.slides, start=1):
        # 슬라이드마다 새로 초기화
        texts, tables, images = [], [], []

        # 텍스트/표/이미지 추출
        for sh in slide.shapes:
            if hasattr(sh, "has_text_frame") and sh.has_text_frame:
                txt = "\n".join(p.text for p in sh.text_frame.paragraphs)
                if txt.strip():
                    texts.append(clean_text(txt))

            if sh.shape_type == MSO_SHAPE_TYPE.TABLE:
                tbl = [[clean_text(c.text) for c in r.cells] for r in sh.table.rows]
                tables.append(tbl)

            if sh.shape_type == MSO_SHAPE_TYPE.PICTURE:
                ext = (sh.image.ext or "png").lower()
                path = os.path.join(
                    media_dir,
                    f"slide{idx:02d}_img_{len(images)+1}.{ext}"
                )
                with open(path, "wb") as f:
                    f.write(sh.image.blob)
                images.append(path)

    state["slides"] = slides_out
    state["n_slides"] = len(slides_out)
    state["slide_index"] = 0
    state["page_contents"], state["scripts"], state["audios"], state["videos"] = [], [], [], []
    return state

```

```

# 스냅샷 생성
tmp_state = {
    "pptx_path": state["pptx_path"],
    "work_dir": work_dir,
    "slide_index": idx - 1
}
snap_state = export_slide_as_png(tmp_state)
snap = snap_state["slide_image"]

# 제목 만들기
title = None
try:
    if slide.shapes.title:
        title = clean_text(slide.shapes.title.text)
except Exception:
    title = None

if not title:
    title = texts[0].split("\n")[0] if texts else f"Slide {idx}"

# 결과물 저장
slides_out.append({
    "index": idx,
    "title": title,
    "texts": texts,
    "tables": tables,
    "images": images,
    "snap": snap,
})

state["slides"] = slides_out
state["n_slides"] = len(slides_out)
state["slide_index"] = 0
state["page_contents"], state["scripts"], state["audios"], state["videos"] = [], [], [], []
return state

```


3. 부록 - 전체 코드

[페이지별 내용 생성]

1) 외부 검색 노드

```
def node_search(state: State) -> State:
    idx = state.get("slide_index", 0)
    slides = state.get("slides", [])
    title = ""
    if slides and idx < len(slides):
        title = slides[idx].get("title", "").strip()

    if not title:
        state["cur_search_context"] = ""
        return state

    try:
        results = tavily_tool.invoke({"query": title})

        result_list = results.get("results", [])
        snippets = [r["content"] for r in result_list if r.get("content")]
        state["cur_search_context"] = "\n\n".join(snippets[:3])
    except Exception as e:
        print(f"[검색 오류] {e}")
        state["cur_search_context"] = ""
    return state
```

```
# node_search 단독 테스트
test_state_search = {
    "slide_index": 0,
    "slides": [{"title": "머신러닝 지도학습", "texts": [], "tables": [], "images": [], "snap": None}],
    "prompt": {"style": "핵심 요점 중심", "tone": "친절한 톤"},
}
```

```
test_state_search = node_search(test_state_search)
print("[검색 결과 (cur_search_context)]")
print(test_state_search["cur_search_context"][:300])
```

[검색 결과 (cur_search_context)]

2) 내용 생성 함수

```
def node_generate_text(state: State) -> State:
    idx = state.get("slide_index", 0)
    slides = state.get("slides", [])

    if not slides or idx >= len(slides):
        raise IndexError(f"현재 slide_index가 잘못되었습니다: {idx}")

    cur_slide = slides[idx]

    texts = cur_slide.get("texts", [])
    tables = cur_slide.get("tables", [])
    images = list(cur_slide.get("images", []) or [])
    slide_img = cur_slide.get("snap")

    table_snip = ""
    if tables:
        try:
            table_snip = "\n".join([" | ".join(map(str, r)) for r in tables[0][:6]])
        except Exception:
            table_snip = str(tables[0][:6])

    if slide_img and slide_img not in images:
        images.insert(0, slide_img)

    prompt = state["prompt"]
    search_results = state.get("cur_search_context", "")

    sys_msg = f"""
# 역할 : 너는 발표 슬라이드의 콘텐츠를 기반으로 핵심 내용을 정리하는 전문가다.

# 지침
반드시 제공된 텍스트·표·이미지에 근거해 설명한다.
웹 검색 결과는 보조 참고자료로만 활용하고, 슬라이드 내용과 충돌하면 슬라이드를 우선한다.
결과는 4~6문장 분량의 자연스러운 문단으로 작성한다.
불렛 포인트는 사용하지 않는다.
스타일은 다음 지시를 따른다: {prompt.get('style')}
이미지가 있다면, 이미지에서 무엇이 보이는지와 그것이 텍스트/표의 내용을 보완하는지도 1~2문장 포함한다.
"""

    user_text_parts = []
    user_text_parts.append(f"[슬라이드 제목]\n{cur_slide.get('title', '')}")
    user_text_parts.append(f"[텍스트]\n{texts if texts else '(없음)'}")
    user_text_parts.append(f"[표 요약]\n{table_snip if table_snip else '(없음)'}")

    if search_results:
        user_text_parts.append(f"[웹 검색 참고자료]\n{search_results}")

    user_text = "\n\n".join(user_text_parts)

    human_msg = [{"type": "text", "text": user_text}]
```

```
MAX_IMGS = 3
for image in images[:MAX_IMGS]:
    try:
        data_url = img_to_data_url(image)
        human_msg.append({
            "type": "image_url",
            "image_url": {"url": data_url}
        })
    except Exception:
        pass

response = llm.invoke([
    SystemMessage(content=sys_msg),
    HumanMessage(content=human_msg)
])

state["cur_page_content"] = response.content.strip()
return state
```

3. 부록 - 전체 코드

[강의 스크립트 생성]

```
def node_generate_script(state: State) -> State:
    idx = state.get("slide_index", 0)
    page_content = state.get("cur_page_content", "").strip()
    scripts = state.get("scripts", [])
    prompt = state.get("prompt", {})

    if not page_content:
        raise ValueError(
            f"cur_page_content가 비어 있습니다. "
            f"slide_index={idx}에서 node_generate_text()가 먼저 제대로 실행됐는지 확인하세요."
        )

    prev_scripts = "###".join(scripts) if scripts else "(이전 슬라이드 없음)"

    sys_msg = f"""
너는 PPT 내용을 바탕으로 자연스러운 강의 발표 대본을 작성하는 AI 강사다.

작성 규칙:
60~90초 분량의 발표 대본을 작성한다.
마크다운 기호(*, #, -, 번호 목록 등)는 사용하지 않는다.
실제 사람이 말하듯 자연스럽게 작성한다.
이전 슬라이드와 흐름이 이어지도록 작성한다.
현재 슬라이드 내용에서 벗어나지 않는다.
톤: {prompt.get('tone', '친절하고 명료한 강의 톤')}
"""

    user_msg = f"""
[현재 슬라이드 번호]
{idx + 1}

[이전 슬라이드 스크립트]
{prev_scripts}

[현재 슬라이드 내용]
{page_content}

위 내용을 바탕으로 인트로, 설명, 마무리 흐름이 자연스럽게 이어지는 발표 대본을 작성해주세요.
"""

    response = llm.invoke([
        SystemMessage(content=sys_msg),
        HumanMessage(content=user_msg)
    ])

    state["cur_script"] = response.content.strip()
    return state
```

[음성 변환]

```
def node_tts(state: State) -> State:
    """
    현재 슬라이드 대본(cur_script)을 MP3 음성 파일로 변환하는 노드입니다.
    work_dir가 state에 없으면 자동으로 기본 폴더를 만들어 사용합니다.
    """

    client = OpenAI()

    # 1) 필수 입력값 확인
    script = state.get("cur_script", "").strip()
    if not script:
        raise ValueError(
            "cur_script가 비어 있습니다. "
            "node_generate_script(state)를 먼저 실행한 뒤 node_tts(state)를 실행하세요."
        )

    # 2) voice / work_dir / slide_index 안전하게 가져오기
    voice = state.get("prompt", {}).get("voice", "alloy")
    work_dir = state.get("work_dir", "./step2_output")
    slide_index = int(state.get("slide_index", 0))

    # state에 work_dir가 없었던 경우를 위해 다시 저장
    state["work_dir"] = work_dir
    os.makedirs(work_dir, exist_ok=True)

    # 3) 슬라이드별로 다른 파일명으로 저장
    mp3_path = os.path.join(
        work_dir,
        f"slide{slide_index + 1:02d}_narration.mp3"
    )

    # 4) TTS 음성 생성
    resp = client.audio.speech.create(
        model=TTS_MODEL,
        voice=voice,
        input=script
    )

    # 5) MP3 파일 저장
    with open(mp3_path, "wb") as f:
        f.write(resp.content)

    # 6) state에 오디오 경로 저장
    state["cur_audio"] = mp3_path

    # 7) MP3 길이 확인
    duration = fprobe_duration(mp3_path)
    print("MP3 길이:", duration, "sec")
    print("저장 경로:", mp3_path)

    return state
```

[영상 제작]

```
def node_make_video(state: State) -> State:
    idx = state["slide_index"]

    s = state["slides"][idx]
    img = s["snap"]
    audio = state["cur_audio"]

    out_mp4 = os.path.join(
        state["work_dir"],
        f"slide{idx + 1:02d}_lecture.mp4"
    )

    state["cur_video"] = render_mp4(img, audio, out_mp4)

    print(f"--- [영상 생성] {idx + 1}번 슬라이드 MP4 생성 완료: {state['cur_video']} ---")

    return state
```


3. 부록 - 전체 코드

[영상 결합]

• State의 키(리스트)에 누적

```
def node_accumulate_result(state: State) -> State:
    """
    [누적 노드] 현재 슬라이드의 결과를 리스트에 저장하고 다음 슬라이드로 인덱스를 넘깁니다.
    """
    # 1. 자료 준비: 현재 슬라이드에서 생성된 임시 데이터를
    cur_content = state.get("cur_page_content", "")
    cur_script = state.get("cur_script", "")
    cur_audio = state.get("cur_audio", "")
    cur_video = state.get("cur_video", "")

    # 2. 요리하기: 전체 누적 리스트(Key)에 추가 (Append)
    # 리스트가 None일 경우를 대비해 get()의 기본값을 빈 리스트로 설정
    state["page_contents"].append(cur_content)
    state["scripts"].append(cur_script)
    state["audios"].append(cur_audio)
    state["videos"].append(cur_video)

    # 3. 결과 담기: 다음 슬라이드를 가리키도록 인덱스 1 증가
    state["slide_index"] += 1

    print(f"--- [누적] {state['slide_index']}번 슬라이드 완료 (총 {state['n_slides']}개) ---")

    # 4. 켄반 넘기기
    return state
```

• 분기 함수

```
def should_continue(state: State):
    """
    [분기 함수] 슬라이드가 남았다면 다시 루프를 돌고, 다 끝났다면 함치기로 보냅니다.
    """
    # 우리가 검토한 slide_index < n_slides 로직 적용
    # 예: 3장일 때 index가 0->1->2까지는 계속(continue), 3이 되는 순간 종료(end)
    if state["slide_index"] < state["n_slides"]:
        print(">>> 다음 슬라이드 처리를 시작합니다.")
        return "continue"
    else:
        print(">>> 모든 슬라이드 처리가 완료되었습니다. 최종 합성을 시작합니다.")
        return "DONE"
```

• 영상 결합

```
import os

def node_concat_videos(state: State) -> State:
    """
    [합치기 노드] 누적된 영상 리스트를 FFmpeg를 사용하여 하나로 병합합니다.
    """
    # 1. 입력 자료 준비
    video_list = state.get("videos", [])
    work_dir = state.get("work_dir", ".")

    # 영상이 없으면 바로 반환
    if not video_list:
        print("!!! 합칠 영상 파일이 없습니다.")
        return state

    # 2. 출력 경로 설정
    final_out_path = os.path.join(work_dir, "final_lecture_video.mp4")

    # 3. 처리: 제공된 concat_videos_ffmpeg 함수 호출
    try:
        print(f"--- [최종 병합] {len(video_list)}개의 영상을 합치는 중... ---")

        # 제공해주신 함수 사용 (reencode=True로 하면 호환성이 좋아지지만, False가 훨씬 빠릅니다)
        concat_videos_ffmpeg(video_list, final_out_path, reencode=False)

        # 4. 결과 저장: 최종 영상 경로를 State에 기록
        state["final_video"] = final_out_path
        print(f"--- [성공] 최종 영상이 생성되었습니다: {final_out_path} ---")

    except Exception as e:
        print(f"!!! 영상 합치기 중 에러 발생: {e}")

    return state
```

